Shoulder-Targeted Pull-Over as a Minimum Risk Maneuver in Autoware: Architecture Study, Extension Design, and a Constrained Classical Trajectory-Generation Framework

Autoware + CARLA Shoulder Detection Project

Framework-phase report — risk/uncertainty integration deferred to a follow-up phase

2026-08-04

Contents

1 Introduction and Scope

1.1 Motivation

The project has, to date, produced a working shoulder *perception* pipeline: a DINOv2-based semantic segmentation node (`shoulder_detection_ros`) that classifies the drivable shoulder in the front camera image, and a LiDAR-camera fusion node (`shoulder_centerline`) that grounds that 2D mask into a smooth, ego-relative sequence of 3D waypoints running down the middle of the shoulder. Both are stable and considered "good enough for now" by the project (see the `shoulder_detection_ws` git history, commit `0f2a02b` and earlier).

The next objective is to *consume* that centerline for an actual safety function: when Autoware's Minimum Risk Maneuver (MRM) system determines the vehicle can no longer continue its normal Dynamic Driving Task (DDT), instead of the two behaviors Autoware currently ships (decelerate to a stop *in the current lane*, either comfortably or in an emergency), the vehicle should be able to change lanes – from whatever lane it currently occupies – toward the shoulder, and park at a lateral offset chosen from the live centerline, without colliding with surrounding traffic.

This document is the output of a dedicated research phase: (1) a structural study of the relevant parts of Autoware Universe (behavior planning, MRM/command-mode arbitration, and the control/validation stack) carried out directly against this project's pinned checkout under `~/autoware/src/universe/autoware_universe`; (2) a literature review restricted to IEEE-venue and otherwise high-credibility sources (no MDPI, per this project's standing citation policy); and (3) a resulting architecture and mathematical framework for a new, Autoware-plugin-based pull-over-to-shoulder MRM.

1.2 Explicit Scope Boundary

Per direction, this phase targets the *classical* formulation only: a deterministic, constrained trajectory-generation and lane-sequencing framework built from the same primitives Autoware's own planner/controller stack already uses (Frenet-frame polynomial trajectories, lanelet-graph routing, MPC/PID tracking). Probabilistic risk and uncertainty modeling – quantifying *how* confident the shoulder detector is at a given point, or reasoning about occluded/unseen hazards – is explicitly deferred to a follow-up phase (§9), by design: every scoring and constraint term

introduced here is structured so that a scalar value today can be replaced by a distribution parameter later without changing the surrounding architecture.

1.3 Document Outline

§2 fixes terminology against the relevant standards. §3 reviews the literature. §4 reports the architecture study findings. §5 makes and justifies the workspace-placement decision. §6 lays out the proposed extension architecture. §7 gives the mathematical formulation. §8 is a self-directed adversarial review, anticipating the objections a methods-track reviewer at a top venue would raise. §9 previews the risk/uncertainty follow-up. §10 and §11 close the document.

2 Standards and Terminology

Two standards anchor the vocabulary used throughout this document:

- **SAE J3016** (*Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles*, SAE International) defines the *Dynamic Driving Task (DDT)*, *DDT fallback*, and the levels of driving automation. An MRM is, in this taxonomy, the mechanism by which an ADS-equipped vehicle performs DDT fallback when the ADS can no longer operate the vehicle within its Operational Design Domain (ODD).
- **ISO 23793-1:2020**, *Intelligent transport systems – Minimal risk manoeuvre (MRM) for automated driving – Part 1: Framework, terms and definitions* [1], is the dedicated MRM standard. It defines the *minimal risk condition* as the stable, low-risk operating condition an ADS reaches at the end of an MRM, and explicitly contemplates multiple MRM strategies (in-lane stop being only one), which is the standards-level hook that justifies treating "pull over onto the shoulder" as a legitimate, standard-anticipated MRM rather than a bespoke deviation from convention.

Autoware's own code reflects this distinction directly: its MRM state machine already carries three named strategies – `comfortable_stop`, `emergency_stop`, and `pull_over` – as first-class members of an enum (§4.2), even though, as this study found, only the first two currently have any behavior wired to them.

3 Related Work

The literature search targeted IEEE Xplore, IEEE Transactions/Journals, and IEEE conference proceedings (ICRA, IROS, ITSC, DSN, DSN-W); MDPI results that surfaced in every query (e.g. an MRM survey in *Vehicles* and a fallback-strategy paper in *Machines*) were deliberately excluded per the project's standing no-MDPI policy, even where topically close, and are not cited here.

3.1 Controllability and Design of Minimal Risk Maneuvers

Vu et al. [2] report a driving-simulator study (35 participants) directly comparing MRM strategies for controllability by human occupants during hand-back/failure transitions. Their central finding – that *a lane change coupled with a standstill on the shoulder lane* was rated the most controllable strategy, ahead of an in-lane stop – is the closest thing in the literature to a direct empirical justification for this project's premise: shoulder pull-over is not merely a nicety, it is the human-factors-preferred MRM outcome when it is geometrically available.

3.2 Emergency and Pull-Over Trajectory Generation

Lee et al. [3] (IEEE Access) present an emergency pull-over algorithm for Level-4 AVs built on model-free adaptive feedback control with online sensitivity estimation and a learning-based gain adaptation layer, targeting exactly the "bring the vehicle to a controlled stop outside the travel lane after a fault" problem. It is evidence that pull-over-as-MRM is an active, IEEE-published research target, and its use of an adaptive/learning correction layer on top of a classical feedback controller is a useful precedent for how this project's Phase 2 uncertainty layer could sit on top of the classical controller retained here.

3.3 Optimal Trajectory Generation in the Frenet Frame

Werling et al. [4, 5] introduced the now-standard approach of decoupling planning into independent longitudinal and lateral 1-D problems in a road-aligned (Frenet) frame, each solved in closed form as a quintic (position-to-position) or quartic (velocity-keeping) polynomial minimizing a jerk-and-time cost functional, followed by re-projection to Cartesian space and a cheap feasibility/collision filter over a sampled family of candidates. This is the exact structure adopted for the trajectory layer of the proposed framework (§7), both because it is the field's reference method for structured-road maneuver generation and because Autoware's own `autoware_behavior_path_goal_planner_module` already uses geometrically simpler variants of the same shift/arc idea for its existing pull-over paths – adopting the full Frenet-polynomial form is a principled generalization of what is already running in this codebase, not a foreign import.

3.4 Contingency and Risk-Aware Planning

Two recent IEEE-Transactions papers define the state of the art this project's Phase 2 will need to engage with. Lin et al. [6] formulate contingency-aware spatiotemporal trajectory optimization that maintains a nominal and a fallback trajectory simultaneously so that a late-revealed hazard does not force an infeasible last-instant maneuver. Yang et al. [7] give a non-conservative contingency-planning method using online-learned, reachable-set-based barrier constraints, solved via consensus ADMM, that avoids the classic over-conservatism of worst-case reachability while retaining a formal safety argument. Both are directly relevant to a future version of the shoulder-goal scoring function in §7.1, which today uses point estimates but is structured to accept exactly this kind of reachable-set or contingency-trajectory reasoning later.

3.5 Formal Safety Framing

Shalev-Shwartz, Shammah, and Shashua's Responsibility-Sensitive Safety (RSS) model [8] is cited here as the SOA formal framework for interpretable, provable-by-construction safety envelopes (longitudinal/lateral safe-distance rules derived from worst-case Newtonian bounds on other agents), and is noted because Autoware's own `path_safety_checker` utilities – reused directly by `goal_planner`, `start_planner`, and (per §4) available to a new pull-over module for free – already implement an RSS-style safety check (`checkSafetyWithRSS`, see §4.1). This is flagged as an arXiv/industry preprint rather than a peer-reviewed IEEE venue, included as foundational SOA context precisely because it is the framework the codebase itself already implements, not as a substitute for a peer-reviewed citation.

3.6 Map Representation

Poggenhans et al. [9] introduce Lanelet2, the HD-map framework Autoware uses throughout its routing and behavior-planning stack, including the `road_shoulder` lanelet subtype that `goal_planner` already queries. Any new module that reasons about "which lane is adjacent to the shoulder" or "how many lane changes separate the current lane from the shoulder" is, by construction, reasoning over this same Lanelet2 routing graph.

4 Autoware Architecture Study

This section reports the findings of a direct source-level study of the project's pinned Autoware checkout (paths rooted at `src/universe/autoware_universe/`), covering the three subsystems a custom pull-over-to-shoulder MRM must interoperate with: the behavior-path-planning plugin system and its existing pull-over/pull-out modules, the MRM/command-mode arbitration pipeline, and the control/validation contract a generated trajectory must satisfy.

4.1 The Behavior Path Planner Plugin System

Autoware's `behavior_path_planner` is a genuine runtime plugin host, not a fixed pipeline. Each maneuver (*scene module*, e.g. lane change, avoidance, goal_planner) is a `pluginlib` class loaded by fully-qualified name:

- A module's *manager* subclasses `SceneModuleManagerInterface` (`autoware_behavior_path_planner_commo` implementing `init()`, `createNewSceneModuleInstance()`, `updateModuleParams()`, and simultaneous-execution predicates.
- The module logic itself derives from `SceneModuleInterface` (same package, `scene_module_interface.hpp`), whose key hooks are `isExecutionRequested()`, `isExecutionReady()`, `updateData()`, `onEntry()`/`onExit()` `plan()`, `planWaitingApproval()`, `planCandidate()`, and the state-machine exit predicates `canTransitSuccessState()`/`canTransitFailureState()`.
- Registration is declared in a package-root `plugins.xml` (e.g. `autoware_behavior_path_goal_planner_module` and the manager node loads modules named in the `launch_modules` ROS parameter, itself assembled by the launch tree under `tier4_planning_launch`.
- Scheduling and simultaneous-execution priority across modules is config-driven via *slots* in `autoware_behavior_path_planner/config/scene_module_manager.param.yaml`; `goal_planner` today sits in a slot ordered after `lane_change`, which is exactly the precedent a new shoulder-pull-over module would follow.

Consequence: a new maneuver can be added as a fully independent sibling package – a new pluginlib class, a `plugins.xml` entry, and a slot-config addition – with *zero edits to core Autoware C++*. This is not a hypothetical; it is exactly how `goal_planner` and `start_planner` themselves are wired in, and it is the mechanism this project's new module will use.

4.1.1 How goal_planner Already Solves an Adjacent Problem

`autoware_behavior_path_goal_planner_module` (˜9,700 LOC across 34 files) is the closest existing analog and a substantial source of reusable machinery:

- It generates candidate pull-over paths using five interchangeable planner types – `SHIFT`, `BEZIER`, `ARC_FORWARD`, `ARC_BACKWARD`, and `FREESPACE` (the last delegating to `autoware_freespace_planning_` for obstructed spots) – each a concrete class under `pull_over_planner/`.
- Candidate *goal poses* are sampled along the target lane by `goal_searcher.cpp`; the target lane is explicitly permitted to be a `road_shoulder`-subtype lanelet (`route_handler.cpp`'s `isShoulderLanelet()` checks the Lanelet2 `Subtype` attribute directly), and `goal_planner` calls `getShoulderLaneletSequence()` to build the parking-maneuver lane sequence.
- Candidates are filtered/ranked by remaining-distance feasibility (`hasEnoughDistance()`) and a genuine safety check, `isSafePath()`, that runs either an RSS-style check (`checkSafetyWithRSS()`) or an integral predicted-occupancy-polygon check against tracked dynamic objects, using the same `path_safety_checker` utilities shared with `lane_change` and `avoidance`.

- **Critical limitation**: `goal_planner`'s own candidate search stays within the current lane sequence plus its *immediate* left/right neighbor. It does not itself induce a multi-lane-change maneuver; `isExecutionRequested()` requires the goal to already be reachable from the current lane within a bounded request length. Reaching a shoulder that is two or more lanes away therefore requires the separate `lane_change` module to run first, with the two coordinated purely through the slot/priority mechanism above – there is no bespoke multi-module orchestration logic to reverse-engineer; the existing scheduler already handles it.
- **Output contract**: every module's `plan()` returns a `BehaviorModuleOutput` containing a `PathWithLaneId` (points tagged with lanelet IDs, for downstream velocity/right-of-way lookups), a `DrivableAreaInfo`, and an optional `modified_goal`. This is what any new module must populate correctly to remain compatible with `behavior_velocity_planner` and the trajectory-generation stages further downstream.
- **Triggering is geometry-driven, not service-driven**: `isExecutionRequested()` activates purely because the *route's own goal pose* falls in/near the shoulder, gated by an `allow_goal_modification` flag carried on the route itself. There is no dedicated runtime API to hand `goal_planner` an arbitrary pose mid-drive. The idiomatic way to trigger it dynamically is therefore *upstream*, at the routing layer: call `autoware_mission_planner_universe`'s AD API service (`SetRoutePoints` / `SetWaypointRoute`) with a new goal at the desired shoulder coordinate and `allow_goal_modification=true`; `route_handler` then updates, and `isExecutionRequested()` naturally goes true on the next planning cycle. This is the mechanism the proposed framework uses (§6) – no hand-written activation logic is needed.

Given this structure, a new shoulder-specific module can realistically reuse an estimated 60–70% of `goal_planner`'s scaffolding (manager/plugin boilerplate, the safety-check call chain, and shift/arc path-shape primitives), writing fresh only the shoulder-centerline-specific goal-scoring logic (§7.1) and any bespoke entry-path shaping the live centerline demands.

4.2 MRM and Command-Mode Arbitration

The checkout contains *two parallel, coexisting* architectures for turning "something is wrong" into "an MRM behavior executes," both fed from the same underlying diagnostic-graph fault classification.

4.2.1 Legacy Path: `autoware_mrm_handler`

`autoware_diagnostic_graph_aggregator` republishes fault state as `tier4_system_msgs/OperationModeAvaila` (booleans: `stop`, `autonomous`, `emergency_stop`, `comfortable_stop`, `pull_over`). `autoware_mrm_handler` (631+179 LOC) subscribes to this on a 10 Hz timer, runs an internal MRM state machine, and dispatches the chosen behavior via `tier4_system_msgs::srv::OperateMrm` service calls to `~/output/mrm/{pull_over,comfortable_stop,emergency_stop}/operate`. Result state is published on `/system/fail_safe/mrm_state` (`autoware_adapi_v1_msgs::msg::MrmState`).

4.2.2 New Path: `command_mode_decider` / `command_mode_switcher`

A newer, superseding architecture unifies operation-mode and MRM handling. `CommandModeDeciderBase` consumes `CommandModeAvailability`/`CommandModeStatus` directly and calls a pluggable `DeciderPlugin::deci` each tick, publishing a `CommandModeRequest` to `command_mode_switcher`, which activates the matching pluginlib-loaded `CommandPlugin`. This is the architecture the proposed framework targets, since it is where active development is happening and where the `pull_over` extension point already has priority wiring (below).

4.2.3 Key Finding: `pull_over` Is a Reserved, Unimplemented Slot

This is the single most consequential finding of the architecture study, and it reframes the whole project: **there is no working pull-over MRM behavior anywhere in this Autoware checkout**, in either architecture –

- In the legacy path, `mrm_handler` already has complete state-machine wiring for `pull_over` (the `use_pull_over` parameter, the `/system/mrm/pull_over_manager/status,operate` topics/service, and dedicated transitions in `updateMrmState()`/`operateMrm()`) – but **no `pull_over_manager` package exists in the source tree at all.** The service client would simply time out if `use_pull_over` were ever set `true` (it defaults `false`).
- In the new path, `PullOverSwitcher` (`autoware_command_mode_switcher_plugins`) is registered in `plugins.xml`, assigned mode ID **2003** (`autoware_command_mode_types/modes.hpp`), and the default decider plugin (`command_mode_decider.cpp`) already gives it the *top* MRM fallback priority – ahead of `comfortable_stop` and `emergency_stop` – **but `PullOverSwitcher::initialize()` is a literally empty function body.** It satisfies the plugin interface and does nothing else: no publishers, no subscribers, no logic.

In other words, this project is not being asked to override or fight existing pull-over behavior; it is being asked to *fill an intentionally reserved, currently-empty extension point* that Autoware's own architects have already prioritized above comfortable-stop and emergency-stop in the fallback chain. That materially changes the risk profile of this work: there is no legacy behavior to regress, and the priority the maintainers assigned to `pull_over` independently corroborates the human-factors finding in [2] that a shoulder pull-over is the preferred MRM outcome when available.

4.2.4 Arbitration Details Relevant to Integration

`CommandPlugin::source()` returns one of `sources::{stop,main,local,remote, emergency_stop}`. `comfortable_stop`, `stop`, `autonomous`, and, critically, `pull_over` all share `sources::main` – meaning they all act *through the normal planning→control→`vehicle_cmd_gate` pipeline*, differentiated only by what they feed into it (a velocity limit, in `comfortable_stop`'s case). Only `emergency_stop` has its own dedicated source, because its legacy operator bypasses planning entirely and writes `Control` commands directly. **This is an architectural constraint on the proposed design**: because `pull_over` is already assigned `sources::main`, an implementation must act by modifying the normal planning pipeline's inputs (route/goal, velocity limits) – not by emitting raw control commands the way emergency-stop does. This is precisely what §6 proposes.

The priority order itself (`pull_over > comfortable_stop > emergency_stop` when more than one is available) is presently **hardcoded** in `CommandModeDecider::decide()`, with an explicit `TODO` comment in that function noting it should eventually follow the AD API fail-safe state-transition specification rather than a fixed order. This is flagged explicitly in §8 and §9 as the exact seam where a future risk/uncertainty-aware arbitration decision ("is a pull-over actually safer than an in-lane stop *right now*, given current traffic?") must eventually be inserted.

No MRM-specific code path was found in `behavior_path_planner` or `planning_validator` – both `comfortable_stop` implementations act purely by tightening the normal planning pipeline's velocity limit (`VelocityLimit` messages), with the rest of the stack (behavior planning, obstacle avoidance, validation) continuing to run unmodified underneath. This confirms the intended integration pattern for `pull_over` is likewise to *reuse*, not bypass, the normal behavior-planning and control stack.

4.3 Control and Trajectory-Validation Contract

Any trajectory this project generates must be consumable by the existing, unmodified control stack.

4.3.1 Trajectory Message Contract

`autoware_trajectory_follower_node` consumes `autoware_planning_msgs::msg::Trajectory` (a `TrajectoryPoint[]`, each carrying `pose` (position *and orientation* – heading must be encoded, not inferred), `longitudinal_velocity_mps`, `lateral_velocity_mps`, `acceleration_mps2`, `heading_rate_rps`, and front/rear wheel angle), synchronized each control cycle with odometry, a steering report, acceleration, and operation-mode state – all five must arrive or the cycle is skipped.

4.3.2 MPC Lateral Controller

Default vehicle model is the kinematic bicycle model with first-order steering delay (3-state: lateral error, heading error, steer). The QP cost combines a tracking-error weight matrix Q (lateral error, heading error, plus terminal weights) with a control-effort weight matrix R (steering input, lateral jerk, steer rate), with a separate low-curvature weight set that switches in below a configured curvature threshold. Default gains are tuned for highway-speed ($<$40 km/h) operation on a specific reference vehicle and are **explicitly expected to need re-tuning** for a slow, high-curvature parking-style entry maneuver – the curvature-indexed weight-set switch in particular is not designed for a regime where curvature is high *everywhere* along the path, which the proposed framework addresses via a dedicated low-speed parameter profile (§7.4).

4.3.3 PID Longitudinal Controller

A four-state FSM (`DRIVE`→`STOPPING`→`STOPPED`, plus `EMERGENCY`). Critically, **the controller does not smooth the reference velocity profile itself** – its own documentation states the trajectory must already carry a smoothed, monotonically-decelerating velocity/acceleration profile down to zero at the stop pose. An `EMERGENCY` transition is triggered by a stop-point overshoot beyond 1.5 m, which is a hard constraint the trajectory generator must respect by construction (§7.2).

4.3.4 Planning Validator

`planning_validator` is the last gate before control, and **any trajectory this project emits must pass through it unmodified in its role, i.e. its numeric thresholds are binding design constraints, not suggestions**: bounds on curvature, relative angle, lateral acceleration ($\leq 9.8\,\mathrm{m/s^2}$), lateral jerk ($\leq 7.0\,\mathrm{m/s^3}$), longitudinal acceleration ($\leq 9.8\,\mathrm{m/s^2}$), steering angle/rate, deviation from ego, and – importantly for a re-planned pull-over path that must update frame to frame – a *consecutive-trajectory shift* limit (lateral 0.5 m, forward 1.0 m, backward 0.1 m at the nearest point) that a jittery replanned path could easily violate if not explicitly smoothed between updates. On failure, the validator does *not* itself raise an MRM; it either passes the trajectory through unchanged, substitutes the last valid trajectory, or replays the last valid trajectory with an overlaid soft-stop deceleration – so a design that reliably fails validation degrades to a stop, not a silent hazard, which is a useful passive safety property to inherit for free.

4.3.5 Lane-Crossing Collision Checking Is Already Available

Two `planning_validator` plugins are directly relevant to a lane-changing pull-over. `rear_collision_checker` is explicitly built for right/left-turn and lane-merge scenarios, combining a blind-spot time-to-collision sub-check and an adjacent-lane RSS-metric sub-check against tracked objects – this is essentially the exact collision-safety net a multi-lane shoulder pull-over needs, and it requires *no new code*, only for the maneuver to actually cross lanes through the normal planning pipeline where this checker already runs. `intersection_collision_checker` is turn-lanelet-specific and likely does not engage for a generic shoulder entry unless that lanelet happens to be turn-tagged.

5 Workspace Architecture Decision

Decision: this work is placed in a new, dedicated overlay workspace, `~/shoulder_pullover_ws`, separate from `shoulder_detection_ws`.

5.1 Rationale

1. **Dependency-weight asymmetry.** `shoulder_detection_ws`'s two packages depend only on core ROS 2, PCL, OpenCV, and tf2 – confirmed by direct inspection of their `package.xml` files; neither links against any Autoware planning/control package. A pull-over MRM module, by contrast, must build against `autoware_behavior_path_planner_common`, `autoware_command_mode_switch` (+ `_types`), `autoware_route_handler`, `path_safety_checker`, and `autoware_planning_msgs` – a substantial slice of the Autoware Universe dependency graph. Folding that into the perception workspace would force every future `shoulder_detection_ws` rebuild to carry that weight and make its build brittle to Autoware core version bumps, for no benefit to the perception code.

2. **Blast-radius and churn isolation.** `shoulder_detection_ws` is currently a stable, signed-off deliverable ("we are okay with this version for now"). The centerline node's own history (§in `shoulder_centerline_node` project memory) shows this class of work churns heavily – multiple full reverts during iteration. A new planning/control-adjacent module, touching safety-relevant interfaces, should be free to iterate and even break without any risk of destabilizing the already-accepted perception deliverable's build or git history.

3. **Only a topic/service coupling is actually needed.** The new module's only runtime dependency on `shoulder_detection_ws` is the `shoulder_centerline_path` topic (and, for triggering, a call to `mission_planner`'s routing service) – both pure ROS 2 interfaces that cross workspace boundaries for free. There is no shared build artifact or header that would require co-location.

4. **Matches both Autoware's own convention and the user's existing operational pattern.** Autoware itself separates perception, planning, control, and system concerns into distinct top-level trees that interoperate purely over topics/services. The user's own launch invocation already layers three overlays (`/opt/ros/humble` → `~/autoware/install` → `~/shoulder_detection_w` a fourth overlay, `~/shoulder_pullover_ws/install`, slots into that same, already-familiar sourcing chain with no change in operational habit.

5.2 Non-Goals

This decision does not propose forking, vendoring, or patching any file inside `~/autoware`. Every extension point identified in §4 is reachable from new, out-of-tree sibling packages (new pluginlib classes, new `plugins.xml` entries, new parameter files referenced from launch), consistent with how `goal_planner` itself is wired in.

6 Proposed Framework Architecture

6.1 Package Layout

Two new ROS 2 packages, both under `~/shoulder_pullover_ws/src/`:

- `autoware_shoulder_pullover_module` – a `behavior_path_planner` scene-module plugin (§4.1's pattern), responsible for: scoring candidate points along the live `shoulder_centerline_path` (§7.1), issuing the routing request that triggers the maneuver, and generating the final shoulder-entry path segment. It deliberately reuses `goal_planner`'s `SHIFT`/`ARC` pull-over-planner primitives and `path_safety_checker` rather than reimplementing them.

- `autoware_shoulder_pullover_switcher` – fills the empty `PullOverSwitcher::initialize()` body (§4.2) with the minimal logic needed to (a) observe MRM-trigger state, (b) hand off to `autoware_shoulder_pullover_module` by driving a routing request, and (c) report `MrmState` transitions (`Operating`→`Succeeded`/`Failed`) back through the existing `command_mode_decider/switcher` channel, mirroring exactly how `ComfortableStopSwitcher` already does this for its behavior. Whether to author a genuinely new class here or directly complete `PullOverSwitcher` in place is left as an open implementation-time decision; both are architecturally equivalent since the mode ID (2003) and priority are already reserved.

6.2 End-to-End Data Flow

1. `shoulder_centerline` (existing, `shoulder_detection_ws`) continuously publishes `~/shoulder_centerline` (ego-relative) and `~/shoulder_centerline_path_map` (map-frame, accumulating).
2. `autoware_shoulder_pullover_module` continuously (not only during an MRM) maintains a scored list of *candidate safe shoulder goals* along the map-frame centerline (§7.1) – precomputing this before it is needed keeps the MRM activation path itself latency-critical-free.
3. On MRM trigger, `autoware_shoulder_pullover_switcher`'s `initialize()`-installed logic is invoked by `command_mode_decider` (already top MRM priority, §4.2); it selects the current best-scored candidate and calls `autoware_mission_planner_universe`'s `SetRoutePoints` service with that pose and `allow_goal_modification=true`.
4. `route_handler` updates; the existing `lane_change` module(s) activate as needed (one activation per lane crossed, sequenced by the existing slot/RTC mechanism – no new coordination logic is written, §4.1) to bring the vehicle into the lane adjacent to the shoulder.
5. `autoware_shoulder_pullover_module`'s own `isExecutionRequested()` goes true once the goal is reachable from the (now shoulder-adjacent) current lane, exactly mirroring `goal_planner`'s own activation condition; it generates the final entry segment using the reused `SHIFT`/`ARC` planner primitives, constrained per §7.2.
6. The resulting `PathWithLaneId` flows through the *unmodified* downstream stack: `behavior_velocity_planne` → obstacle-avoidance/path-optimizer → `planning_validator` (§4.3, all thresholds respected by construction) → `trajectory_follower_node` (MPC + PID, switched to a low-speed parameter profile, §7.4) → vehicle.

6.3 Reuse Ledger

Reused as-is	**New**
`path_safety_checker` (RSS + predicted-polygon)	Shoulder-centerline goal scoring
`lane_change` module + slot scheduler	`PullOverSwitcher` MRM-trigger logic
`goal_planner`'s `SHIFT`/`ARC` path primitives	Low-speed MPC/PID param profile
`rear_collision_checker` (adjacent-lane RSS/TTC)	Frenet constraint set from §7.2
`mission_planner`'s `SetRoutePoints` routing API	
`planning_validator` (all checks, unmodified)	

Table 1: What the framework reuses wholesale from Autoware core versus what is genuinely new.

7 Mathematical Formulation and Methodology

The formulation below is deliberately *classical* (deterministic point estimates, hard constraints) per the stated scope, but every term is annotated where it is designed to become a distribution or a reachable-set bound in Phase 2 (§9).

7.1 Safe Shoulder-Goal Scoring

Let $\mathcal{W} = \{w_1, \ldots, w_N\}$ be the map-frame waypoints of `shoulder_centerline_path_map` at query time, each carrying position, heading, signed curvature $\kappa(w_i)$, and (from the underlying `world_bins_` accumulation) the number of supporting samples $n(w_i)$ as a maturity proxy. Define a candidate score

$$S(w_i) = \alpha_1 R(w_i) \; + \; \alpha_2 M(w_i) \; + \; \alpha_3 C(w_i) \; - \; \alpha_4 D(w_i) \; - \; \alpha_5 (w_i) \tag{1}$$

with all terms normalized to $[0, 1]$ and $\sum \alpha_j = 1$:

- $R(w_i)$ – *continuous safe run length* ahead of w_i: the arclength over which consecutive waypoints remain classified/confirmed (bounded by $n(w_j) \geq n_{\min}$ for a forward window), rewarding a goal with enough confirmed shoulder *after* it to actually stop, not just at it. *(Phase 2: replace the hard $n_{\min}$ gate with the probability the run remains valid under detector uncertainty.)*
- $M(w_i) = n(w_i)/n_{\text{ref}}$ – maturity: how many independent LiDAR/mask-IPM samples back this bin, capped at a reference count; directly reuses the existing `min_world_bin_samples` confirmation mechanism (currently 44, see project memory) as a ready-made quality signal.
- $C(w_i)$ – lateral clearance margin, from the extent-midpoint estimator's own trimmed min/max edge distances, rewarding goals nearer the center of a *wide* confirmed shoulder segment over a narrow one.
- $D(w_i)$ – normalized ETA/distance from current ego pose along the route, penalizing candidates that would require an infeasibly abrupt maneuver.
- $(w_i) = |\kappa(w_i)|/\kappa_{\max}$ – normalized centerline curvature, penalizing goals on a sharply curved stretch of shoulder (harder to enter cleanly at low speed).

The selected goal is $w^\star = \arg\max_i S(w_i)$ subject to $D(w_i) \geq D_{\min}$ (a hard minimum-stopping-distance floor derived from the current speed and the comfortable/emergency deceleration limits already configured for `comfortable_stop`, so the maneuver is never requested somewhere the vehicle physically cannot decelerate to before reaching).

7.2 Constrained Frenet-Frame Trajectory Generation

Following [4, 5], the entry maneuver from the shoulder-adjacent lane's centerline to $w^\star$ is planned in that lane's Frenet frame (s, d), decoupled into independent lateral and longitudinal problems.

Lateral (quintic) problem. Given boundary conditions $(d_0, \dot{d}_0, \ddot{d}_0)$ at $t = 0$ and $(d_f, 0, 0)$ at $t = T$ (a clean stop laterally centered on the shoulder), the unique minimum-jerk polynomial is

$$d(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5, \tag{2}$$

with coefficients solved in closed form from the six boundary conditions (standard quintic Hermite solve, as in [4]). A family of candidates is generated by sampling (d_f, T) over a bounded grid, and each is scored by

$$J_d = k_j \int_0^T \dddot{d}\,(t)^2 \, dt \; + \; k_T \, T \; + \; k_d \, (d_f - d^\star)^2, \tag{3}$$

the classic jerk/time/terminal-offset trade-off, where $d^\star$ is the lateral offset to $w^\star$'s position in this frame.

Longitudinal problem. Two regimes are used depending on maneuver phase: a velocity-keeping quartic while still in the travel lane, and a stop-to-position quintic for the final approach

into the shoulder (boundary condition $\dot{s}(T) = 0$, satisfying, by construction, the PID longitudinal controller's requirement in §4.3 that the reference trajectory itself carry a smoothed, monotonic deceleration to zero – the controller is never asked to invent that shape).

Hard constraints. Every sampled candidate is rejected unless it satisfies, for all $t \in [0, T]$:

$$|\kappa(t)| \le \kappa_{\max} \quad \text{(from } \texttt{planning\_validator}\text{'s curvature bound, §4.3)} \tag{4}$$

$$|a_{\text{lat}}(t)| \le 9.8 \text{ m/s}^2 \quad \text{(validator lateral-accel bound)} \tag{5}$$

$$|j_{\text{lat}}(t)| \le 7.0 \text{ m/s}^3 \quad \text{(validator lateral-jerk bound)} \tag{6}$$

$$|a_{\text{lon}}(t)| \le 9.8 \text{ m/s}^2 \quad \text{(validator longitudinal-accel bound)} \tag{7}$$

$$|\delta(t)| \le \delta_{\max},\ |\dot{\delta}(t)| \le \dot{\delta}_{\max} \quad \text{(steering angle/rate bound)} \tag{8}$$

with curvature related to the Frenet derivatives by the standard planar formula $\kappa(t) = \dfrac{\dot{x}(t)\ddot{y}(t) - \dot{y}(t)\ddot{x}(t)}{\left(\dot{x}(t)^2 + \dot{y}(t)^2\right)^{3/2}}$ after re-projection to Cartesian coordinates. **These are not arbitrary safety margins chosen by this project; they are read directly from `planning_validator`'s own configured thresholds** (§4.3), so a generated trajectory is validator-compliant *by construction* rather than by hope, and the frame-to-frame *trajectory-shift* limit the validator also enforces (0.5 m lateral / 1.0 m forward / 0.1 m backward) is additionally imposed as a rate limit on how much $w^\star$ itself is permitted to change between consecutive re-plans (a low-pass filter on the arg max selection in Eq. 1, not just on the resulting path).

Among all constraint-satisfying candidates, the one minimizing the combined cost $J = J_d + J_s$ (lateral plus the analogous longitudinal cost) is selected – a sampling-based constrained optimum, not a full nonlinear program, which is what keeps this tractable in real time (§7.5).

7.3 Multi-Lane-Change Sequencing

The lanelet2 routing graph already maintained by `route_handler` is treated as a directed graph $G = (V, E)$ over lanelets, and the lane sequence from ego's current lane ℓ_0 to the shoulder-adjacent lane ℓ_k is the shortest path (by lane-change count, or by a cost weighting lane-change difficulty) in G. Each edge on that path corresponds to exactly one activation of the existing `lane_change` module, itself Frenet-planned by Autoware core; the final edge into the shoulder lane is the module described in §7.2. No new graph-search code is strictly required – `route_handler`'s existing route-generation already performs this search when `SetRoutePoints` is called with the shoulder goal; this subsection documents the underlying mechanism rather than proposing a new algorithm to write.

7.4 Controller Compatibility

Because default MPC/PID gains are tuned for highway-speed operation (§4.3), the framework proposes a discrete *parameter profile switch* – not a new controller – activated for the duration the maneuver's `MrmState.behavior == PULL_OVER`: tightened Q/relaxed R weighting for tracking accuracy over control smoothness (justified since speed, and therefore disturbance energy, is low), a shortened MPC prediction horizon matched to the maneuver's own short length, relaxed curvature-indexed steer-rate tables (since curvature is deliberately high throughout a shoulder entry, not just in corners), and confirmation that `mpc_min_prediction_length` (5.0 m default) does not exceed the generated segment's own length. This mirrors the precedent `comfortable_stop` already sets of modifying planning/ control *parameters* rather than substituting a new controller implementation.

7.5 Computational Complexity

Goal scoring (Eq. 1) is $O(N)$ in the number of live centerline waypoints (typically $\leq$ a few hundred). Trajectory generation is the standard Werling-style sampling scheme: closed-form quintic/quartic coefficients per candidate ($O(1)$ each, no iterative solve), over a bounded (d_f, T) grid, giving overall $O(|grid|)$ per re-plan cycle – the same complexity class Werling et al. report as real-time-capable on 2010-era hardware [4], and substantially cheaper than the `FREESPACE` (hybrid-A*-class) option `goal_planner` already carries for the rare obstructed case, which remains available as a fallback planner type without any new complexity budget being introduced.

8 Anticipated Critiques and Mitigations

This section is a deliberate self-adversarial pass, anticipating the objections a rigorous reviewer at a top venue (IEEE T-ITS, T-IV, ICRA/IROS) would raise against this framework as described so far.

1. **"A fixed-priority MRM arbitration (pull_over > comfortable_stop > emergency_stop) is not obviously correct."** A shoulder pull-over that takes several seconds and crosses lanes can, in sufficiently dense or erratic traffic, be riskier than an immediate in-lane stop – yet Autoware's own `command_mode_decider` currently selects by a hardcoded, context-blind order (and, per §4.2, its own source code carries a TODO acknowledging this). *Mitigation*: this is explicitly not solved in this phase; it is named as the primary target of the Phase 2 risk layer (§9), which will condition the arbitration decision on live traffic/uncertainty rather than a fixed lexicographic order.

2. **"No formal safety guarantee is offered."** The framework provides constraint satisfaction against Autoware's configured operating limits and reuses RSS-style pairwise safety checks, but offers no system-level guarantee against, e.g., a simultaneous multi-agent worst-case scenario, unlike a full reachable-set or RSS-composed proof. *Mitigation*: explicitly scoped out as future work ([8], [7]); this document does not claim formal safety, only constraint compliance with the existing, already-deployed safety layer.

3. **"The evaluation protocol is unspecified."** No experiment design has yet been run; a single CARLA route on GPU-contended hardware (as used for the perception work) is not statistically meaningful. *Mitigation*: the natural next step is a defined protocol – multiple routes/seeds, a baseline comparison against stock `comfortable_stop` (in-lane stop) and a synthetic "no MRM" control, and quantitative metrics (success rate, lateral deviation from shoulder center at rest, peak/RMS lateral jerk, time-to-stable-stop, and any validator soft-stop/rejection events) – flagged here as required before any performance claim is made, not yet executed.

4. **"The shoulder representation is a simplified spline, not obstacle-instance aware."** The centerline is a smoothed geometric estimate; it does not itself detect discontinuities such as driveways, mailboxes, or a parked vehicle already occupying the target spot. *Mitigation*: partially covered for free, since the final path still passes through `behavior_velocity_planner/path_safety` object-aware collision checks before reaching control (§4.3) – but the *goal-selection* stage itself (Eq. 1) does not yet reason about occupancy at the candidate pose; adding an occupancy/object check at candidate-scoring time (not just at final-path-safety time) is named as an immediate hardening item even within the classical phase, ahead of full Phase 2 work.

5. **"Single monocular camera + LiDAR fusion has known range-dependent geometric bias."** Already diagnosed and partially mitigated in the perception layer itself (the `world_min_capture_x` absolute-distance gate, project memory), reflecting a documented

monocular-IPM limitation, not a new concern for this document – but it means goal candidates necessarily inherit whatever residual bias remains in the centerline at the time of query; the maturity term $M(w_i)$ in Eq. 1 is the direct mitigation, preferring well-confirmed bins over freshly observed ones.

6. **"Domain gap: a DINOv2 model trained on CVAT annotations of CARLA imagery may not transfer to real sensors."** True and unaddressed by this document, which is explicitly scoped to the CARLA-simulated project this codebase currently targets; any claim of real-world applicability would require re-validation with real-world training data and sensor noise characteristics, named here as an explicit limitation (§10), not something this framework solves.

7. **"Retuned low-speed MPC/PID gains are asserted, not derived or validated."** §7.4 states the *direction* of retuning (tighter tracking weight, shorter horizon, relaxed steer-rate tables) from first principles about the operating regime, but offers no tuned numeric values nor closed-loop stability analysis. *Mitigation*: flagged as implementation-phase work requiring either manual tuning against the validator's own numeric limits or a data-driven gain-scheduling approach, not resolved here.

8. **"Reuse estimates (60–70% of goal_planner's scaffolding) are an informed guess, not a measured figure."** Stated honestly as an estimate derived from structural similarity assessed during the architecture study (§4.1), not from having written the new module yet; the real figure will only be known after implementation.

9 Phase 2 Preview: Risk and Uncertainty Integration

Although out of scope for this document, the framework above was deliberately shaped to make the following extensions additive rather than architecturally disruptive:

- **Goal scoring** (Eq. 1): each term R, M, C, D, becomes the mean of a distribution (e.g. a Beta distribution over detector confidence for M, a Gaussian over centerline lateral position for C), and arg max selection becomes an expected-utility or chance-constrained selection, in the style of [6].
- **Trajectory constraints** (§7.2): the deterministic bounds become either chance constraints (probability of violation $\leq \epsilon$) or reachable-set barrier constraints following [7], particularly around occluded regions the perception system cannot currently see past.
- **MRM arbitration**: the hardcoded `pull_over > comfortable_stop > emergency_stop` order in `command_mode_decider` (§4.2) is the concrete, identified seam where a risk-conditioned decision – e.g., an RSS-style [8] or contingency-trajectory-based comparison of the two strategies' realized risk under current traffic – replaces the fixed lexicographic order, directly answering the critique in §8, item 1.

10 Limitations of This Document

This is an architecture and framework document, not an implementation report: no new code has been written or tested against the running stack; all Autoware-side findings are current as of the project's pinned checkout and may drift as upstream Autoware evolves (notably, the legacy `mrm_handler` path appears to be mid-deprecation in favor of `command_mode_decider`, per the coexistence of both architectures observed in §4.2); and no simulation results are yet available to substantiate any of the quantitative claims implied by §7 (e.g. real-time feasibility is argued by complexity class and precedent, not measured on this project's own hardware).

11 Conclusion

The architecture study found that Autoware Universe already reserves, but does not implement, a top-priority `pull_over` MRM behavior in its current command-mode arbitration system – independently corroborating human-factors evidence [2] that a shoulder pull-over is the preferred MRM outcome when geometrically available. This project's existing shoulder-detection and centerline-waypoint perception stack is directly usable to fill that slot. The proposed framework fills it via two new, plugin-only packages in a dedicated overlay workspace, reusing Autoware's own `goal_planner`, `lane_change`, `path_safety_checker`, and `planning_validator` machinery wherever possible, and introduces a classical, Werling-style constrained Frenet-frame trajectory layer whose constraint bounds are drawn directly from the existing control/validation stack's own configured limits, so generated maneuvers are compatible by construction rather than by later tuning. Every scoring and constraint term is explicitly structured for a Phase 2 risk/uncertainty extension, with the highest-value single target for that phase – the currently-hardcoded MRM strategy arbitration order – identified concretely in §4.2 and §8.

References

[1] ISO 23793-1:2020, *Intelligent transport systems — Minimal risk manoeuvre (MRM) for automated driving — Part 1: Framework, terms and definitions*, International Organization for Standardization, 2020.

[2] V. Vu, F. Warg, A. Thorsén, S. Ursing, F. Sunnerstam, J. Holler, C. Bergenhem, and I. Cosmin, "Minimal Risk Manoeuvre Strategies for Cooperative and Collaborative Automated Vehicles," in *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*, 2023. doi: 10.1109/DSN-W58399.2023.00039.

[3] J. Lee, K. Oh, S. Oh, Y. Yoon, S. Kim, T. Song, and K. Yi, "Emergency Pull-Over Algorithm for Level 4 Autonomous Vehicles Based on Model-Free Adaptive Feedback Control With Sensitivity and Learning Approaches," *IEEE Access*, vol. 10, 2022. doi: 10.1109/ACCESS.2022.3156275.

[4] M. Werling, J. Ziegler, S. Kammel, and S. Thrun, "Optimal Trajectory Generation for Dynamic Street Scenarios in a Frenét Frame," in *2010 IEEE International Conference on Robotics and Automation (ICRA)*, 2010, pp. 987–993. doi: 10.1109/ROBOT.2010.5509799.

[5] M. Werling, S. Kammel, J. Ziegler, and L. Gröll, "Optimal Trajectories for Time-Critical Street Scenarios Using Discretized Terminal Manifolds," *The International Journal of Robotics Research*, vol. 31, no. 3, 2011. doi: 10.1177/0278364911423042.

[6] Z. Lin, J. Lan, A.-T. Nguyen, and D. Flynn, "Contingency-Aware Spatiotemporal Optimization for Safe Autonomous Vehicle Trajectory Planning," *IEEE Transactions on Intelligent Transportation Systems*, 2025. doi: 10.1109/TITS.2025.3597234.

[7] R. Yang, L. Zheng, S. Ge, and J. Ma, "Safe and Nonconservative Contingency Planning for Autonomous Vehicles via Online Learning-Based Reachable Set Barriers," *IEEE Transactions on Control Systems Technology*, 2025/2026. doi: 10.1109/TCST.2026.3675339.

[8] S. Shalev-Shwartz, S. Shammah, and A. Shashua, "On a Formal Model of Safe and Scalable Self-Driving Cars," arXiv:1708.06374, 2017.

[9] F. Poggenhans, J.-H. Pauls, J. Janosovits, S. Orf, M. Naumann, F. Kuhnt, and M. Mayr, "Lanelet2: A High-Definition Map Framework for the Future of Automated Driving," in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018. doi: 10.1109/ITSC.2018.8569929.